Document Number:	P0071R00
Date:	2015-09-12
Project:	Programming Language C++, Evolution
Revises:	none
Reply to:	gorn@microsoft.com

P0071R00: Coroutines: Keyword alternatives

Introduction

One of the question raised in Lenexa was: "Can we take a `yield` identifier and make it a C++ keyword?". The answer was resounding no. There was some ad-hoc brainstorming during the session on possible alternatives. This paper examines the alternatives and makes recommendations.

Ideal

We believe that in the world where we could have any keywords we would like, our choice would be `await` and `yield`, as they most clearly capture the notion of `await`-ing on something or `yield`-ing a value to the consumer.

```
  future<int> deep_thought() {        generator<char> hello() {
    await 5ms;                          for(auto ch: "Hello, world")
    return 42;                            yield ch;
  }                                   }
```

But, we don't live in an ideal world. Developers use nice concise words as names of functions, variables, classes, namespaces, etc. Can we keep nice keywords and don't break existing code?

Soft Keywords

Soft Keywords proposal (p0056r0) introduces a new named entity into the language: soft keyword. Soft keywords are predefined in `std` namespace and participate in name lookup and name hiding according to existing language rules with some exceptions listed in the paper.

Making `await` and `yield` soft keywords, not only allows to use `yield` and `await` in coroutines and don't break existing code, it will also allow use of coroutines in software where `await` and `yield` are names of existing functions or classes, since soft keywords allow qualification, as in `std::yield`.

Magic functions

What if `yield` and `await` are magic functions defined in the `std` namespace?
It will make the syntax a little bit noisier, since we now always have to put parenthesis around the expression being yielded or awaited upon.

```
    auto conn = await Tcp::Connect("127.0.0.1", 1337);
    auto bytesRead = await conn.Read(buf, sizeof(buf));

will become

    auto conn = await(Tcp::Connect("127.0.0.1", 1337));
    auto bytesRead = await(conn.Read(buf, sizeof(buf)));
```

If we go one step further and increase the magic by giving magic functions precedence and unary operator call syntax, we just reinvented the soft keywords described in the previous section.

Find a new name for yield

Lowercase `await` is relative rarely used identifier (as compared to yield) and it has a suitable replacement, namely `wait`, which carries roughly the same meaning and spelling. Thus, while taking `await` as a keyword will break some software, having a closely related replacement `wait` will lessen the pain.

`yield` is used more frequently and there is no obvious term to replace it with.

export

For coroutines `yield` means both *yield a value* as in output the value, produce the value as well *yield execution back to the caller*. Out of existing set of C++

keywords only one carries a meaning that is at least somewhat close to output, that is `export`. However this usage of `export` will be confusing as the export is used in the modules (N4465) with different meaning.

```
module greeting;

export generator<char> hello() { // exporting function hello
  for(auto ch: "Hello, world")
     export ch;                  // yielding a character
}
```

On the plus side, it is short and it is already a keyword.

yieldexpr or yield_expr

`yieldexpr` and `yield_expr` are sufficiently odd identifiers and score no hits on github or codesearch.

```
generator<char> hello() {
  for(auto ch: "Hello, world") {
     yieldexpr ch;  // similar to constexpr
     yield_expr ch; // similar to const_cast
  }
}
```

coyield or co_yield

Instead of using a suffix to oddify `await` and `yield` we can look at having an oddification prefix, such as `co_` or `co` as was suggested during Lenexa coroutine discussion.

Without the underscore, `co` prefix leads to wrong visual parsing as in `coy-ield` and thus inferior to `co_`. We also need to make a choice if we want to mangle `await` for symmetry.

```
  future<int> deep_thought() {    generator<char> hello() {        auto xform(Stream& s) {
    await 5ms;                      for(auto ch: "Hello, world")     for await(auto x: s)
    return 42;                        co_yield ch;                     co_yield make_pair(x,time());
  }                               }                                }
  auto operator await(future<void>&) { ... }

or

  future<int> deep_thought() {    generator<char> hello() {        auto xform(Stream& s) {
    co_await 5ms;                   for(auto ch: "Hello, world")     for co_await(auto x: s)
    return 42;                        co_yield ch;                     co_yield make_pair(x,time());
  }                               }                                }
  auto operator co_await(future<void>&) { ... }
```

Given that `await` is used in more contexts, such as name of the operator and an option for a range-based-for statement, keeping a short version `await` as opposed to oddification `co_await` is preferrable.

yield_placeholder

To save committee time on bikeshed, use some obviously bad name, such as `yield_placeholder` and defer decision aboug *yield-keyword* until a more suitable alternative is discovered / invented.

Conclusion

Soft keyword or its less general version magic functions leads to the best coroutine syntax. None of the other alternatives considered come close to the beauty of the clean `await` and `yield` keywords. If at all possible, our overwhelming preference is for selecting one of the first two alternatives or defer the bikeshed using `yield_placeholder` until some better alternatives are invented or we hit a deadline that a keyword must be picked or else the feature will not make the standard.

References

p0056r0: Soft Keywords (http://wg21.link/p0056r0)

N4465: A Module System for C++ (Revision 3) (http://wg21.link/n4465)